Detecting Harmful LLM-Agent Commands in Industrial IoT

Task-Aware Graph Neural Networks

Billal Mokhtari Yulliwas Ameur Samia Bouzefrane

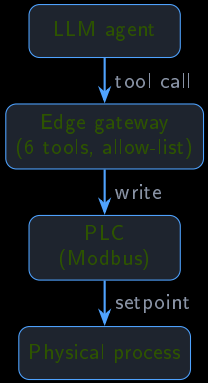
CEDRIC Laboratory, Cnam Paris
BATTwin project

October 2026

IEEE Internet of Things Journal, Special Issue on Graph Representation Learning for IoT

The new situation: LLM agents are driving factories

- An **LLM agent** now assists the human operator of a plant.
- It cannot touch the machines directly. It acts through an **edge gateway** that offers a few tools.
- The gateway turns each tool call into a **Modbus write** to a PLC.
- The agent works with the rights of a **trusted operator**.



The problem in one line

A misled or hijacked agent can send a command that is **valid, authorized and on the allow-list**, yet still harmful.

The hardest command to catch: context-shifted

Everyday analogy.

"Drive at 90 km/h" is:

- fine on the motorway,
- dangerous in front of a school.

Same command. Only the **situation** changed.

In the plant.

Write `coating_speed = 2.40 m/min`:

- steady state: dryer hot, coating dries, good batch;
- dryer warm-up: oven still cold, the film comes out wet, the batch is ruined.

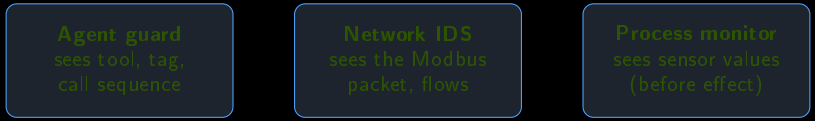
Benign (10:00)

write coating_speed = 2.40
phase = **steady**, dryer 110°C

Harmful (10:40)

write coating_speed = 2.40
phase = **ramp**, dryer 65°C

Why every current defense is blind to it



Observation 1 (single-view blindness)

If a score depends on a command only through its triple $(a, \kappa, \mathrm{bin}(v))$ and agent history, then the benign command and its harmful twin get the **exact same score**
Same for a network view, and for a process view before the effect appears.

Defining the command and the slice

A command is a 4-tuple:

$$c = (a,\ \kappa,\ v,\ t) \qquad \text{tool, target tag, value, time}$$

- $\mathrm{bin}(v)$ maps the value to one of Q **quantile bins** learned on benign data.
- $\mathcal{C}_{\text{benign}}$ = set of triples $(a, \kappa, \mathrm{bin}(v))$ ever seen in benign training.

Definition: context-shifted command

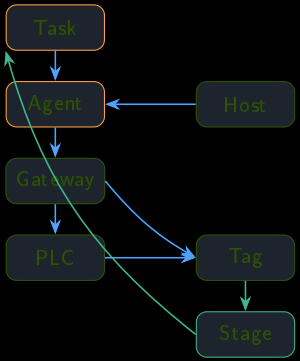
c is context-shifted if all three hold:

$$\underbrace{(a, \kappa, \mathrm{bin}(v)) \in \mathcal{C}_{\text{benign}}}_{\text{looks normal}} \wedge \underbrace{\text{allow-list permits it}}_{\text{passes the gateway}} \wedge \underbrace{\text{oracle} = \text{harmful}}_{\text{actually damages}}$$

This is the exact test slice we report separately.

Our idea: link everything in one graph at the gateway

- Build a **dynamic heterogeneous graph** from the gateway logs and the fieldbus.
- It connects the **task** to the **plant state**, which no single view does.
- A temporal GNN learns, from **benign data only**, how likely each command is in its context.
- Score each command **before** its effect appears.



Task and Stage are the nodes that single-view detectors never connect.

The task–plant graph (schema)

Node types

Node	Key features
Task	type, target stage, tag set
Agent	session age, call rate
Gateway	call rate, error rate
Host	role, flow stats
PLC	request rate
Tag	value, rolling mean/std
Stage	phase, time in phase

Edge types

Edge	Carries
assigned	time since assign.
invokes	tool id
writes/reads	FC, value, delta
flow	NetFlow features
controls, belongs	static

Strict rule

No feature contains a **recipe**, an **operating limit** or a **twin output**. The detector never sees them.

Snapshots over windows $\Delta = 10$ s, stride 5 s.

The model: from graph to probability



Memory carries information across snapshots:

$$\mathrm{m}_v^t = \mathrm{GRU}\big(\mathrm{h}_v^t,\ \mathrm{m}_v^{t-1}\big)$$

- HGT mixes information **across the graph** (neighbours, typed edges).
- GRU mixes information **across time** (recent history of each node).
- A new node starts from a zero memory.

Task-conditioned likelihood and the score

Trained on benign data only, the model predicts the written tag and value bin from the context just before the command:

$$p_\theta\Big(\kappa,\ \mathrm{bin}(v) \;\Big|\; \mathrm{m}_\tau^{t^-},\ \mathrm{m}_{\text{agent}}^{t^-},\ \mathrm{m}_{\text{stage}(\kappa)}^{t^-},\ \mathrm{m}_\kappa^{t^-}\Big)$$

t^- = last snapshot before t, so the effect is never visible.

The anomaly score is the **surprise**:

$$s(c) = -\log p_\theta(\kappa,\ \mathrm{bin}(v) \mid \cdot)$$

- A context-shifted command has a **frequent triple** but an **unlikely context**, so $s(c)$ is high only if the model truly uses the task and the stage.
- Alarm threshold set on benign validation at a **1% false positive rate**.

How we test it honestly (a test that can fail)

Independent attacks

- A **red-team LLM** from a different family generates the attacks.
- It gets only a MITRE ATT&CK for ICS technique + the plant docs.
- No hand-written attack script.

External oracle

- Labels by checking **process outputs** against operating limits.
- The detector **never sees** these limits.

Baselines

- B0 phase-aware value table
- B1 allow-list
- B2 same-feature tabular (XGBoost)
- B3 network view (E-GraphSAGE)
- B4 agent view (AIRGuard)
- B5 LLM judge

Splits: chronological (70/10/20), never random. Thresholds never see red-team data.

The strongest test: an adaptive, effect-preserving attacker

A grey-box attacker tries to stay harmful while lowering our score:

$$\min_{\delta \in \mathcal{R}_B(e)} \; \max_{c \in e \oplus \delta} \; s_\theta(c) \qquad \text{s.t.} \qquad \mathrm{Oracle}(e \oplus \delta) = \text{harmful}$$

- e is an episode, $\mathcal{R}_B(e)$ the **realizable edits** with at most B added events.
- Allowed edits: insert benign reads, interleave old benign commands, split a change, shift in time.
- Every edit must **keep the physical effect** (checked by the oracle).
- Budgets $B \in \{0, 4, 8\}$.

This stops us from claiming robustness we do not have.

Pre-registered hypotheses (they are allowed to fail)

H1 (main)

On context-shifted commands, our recall at 1% FPR beats B0, B2, B3, B4, B5 by at least 10 points (McNemar, Holm-corrected $p < 0.05$).

H2 (where the signal comes from)

Single-view ablations stay at AUROC ≤ 0.6; removing the **task** node hurts more than removing the network subgraph.

H3 (robustness)

At $B = 4$, fewer harmful episodes evade our model than B2 and B4.

If H1 fails against B0, the task adds nothing beyond phase statistics. We report that too.

To fix in the article (1/2): notation and timeline

Notation clash

- τ is used for **two different things**: the task (Sec. IV, m_τ) and the oracle margin ("leaves its limits by at least τ", Sec. V). **Rename one**, e.g. margin $\rightarrow \varepsilon$.

Timeline is now behind

- Fallback-oracle decision was dated **17/10**, OSF registration **22/10**, pilot **25/10**. We are past these.
- Decide the new realistic dates before submission (deadline 30/11).

To fix in the article (2/2): content decisions

- **Placeholders to fill:** the abstract result sentence, all $[X]$ in the tables, model names and sizes (operator and red-team), benign hours and command counts.
- **Final hyperparameters + search grid** (at most 2 values each, chosen on benign validation only), training time, parameter count.
- **Oracle choice** (primary twin replay vs fallback) must be locked, with the limits table in the supplement.
- **H2 claim "AUROC ≤ 0.6"** is strong. Confirm it survives the pilot, or soften it.
- **Figure 1** (architecture) is still a placeholder box: produce the final vector PDF.
- **Scope sentence:** double-check the "emulated vs synthetic" wording is consistent with the dataset paper.

Where we stand and what is next

The contribution

- A named, measurable class: **context-shifted commands**.
- A **task-conditioned likelihood** at the edge, trained on benign data.
- A test that can fail: independent red-team + external oracle + adaptive attacker.

Next steps

- Lock the oracle and the dates.
- Run the pilot and the go/no-go test.
- Run the main campaign.
- Fill the tables and the final figure.

Thank you. Questions and feedback welcome.